OTH Amberg-Weiden

TON-IOT INTRUSION DETECTION

Performance vs. Ressourcenverbrauch

Raphael Balzer und Kevin Paulus

© 2024 OTH Amberg-Weiden

TON-IoT Intrusion Detection

Übersicht

01 Zielsetzung

02 Isolation Forest

03 CatBoost Classifier

04 Support Vector Machine

05 Evaluation

TON-IoT Intrusion Detection

Übersicht

01 **Zielsetzung**

02 Isolation Forest

03 CatBoost Classifier

04 Support Vector Machine

05 Evaluation

Erste Headline

OTH Amberg-Weiden

1. Problemstellung

- **Wandel der Infrastruktur:** Trend von zentralen Servern hin zu Edge Computing
- **Die Herausforderung:** IDS-Algorithmen müssen auf ressourcenbeschränkten Geräten laufen
- **Kernfrage:** Wie finden wir den optimalen Kompromiss zwischen High-Performance und Low-Energy?

2. Zielsetzung der Arbeit

Ein systematischer Vergleich von ML-Modellen (SVM, Isolation Forest, Boosting) basierend auf zwei Säulen:

Detektionsqualität
- Accuracy & Precision
- F1- & F2-Score
- Recall

Ressourceneffizienz
- Inferenzzeit (Latenz)
- Speicherverbrauch (RAM)
- Energieverbrauch

TON-IoT Intrusion Detection

Übersicht

OTH Amberg-Weiden

01 Zielsetzung

02 **Isolation Forest**

03 CatBoost Classifier

04 Support Vector Machine

05 Evaluation

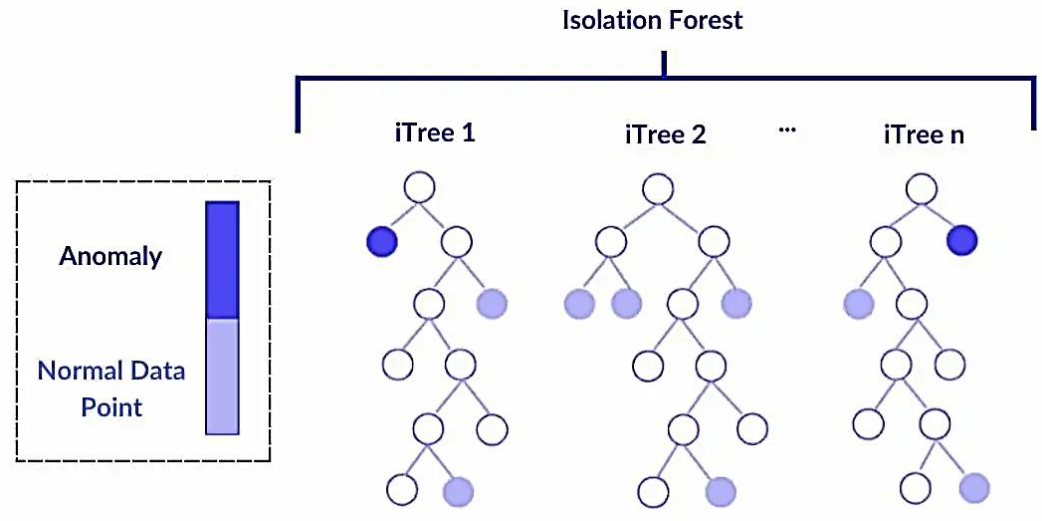
Isolation Forest

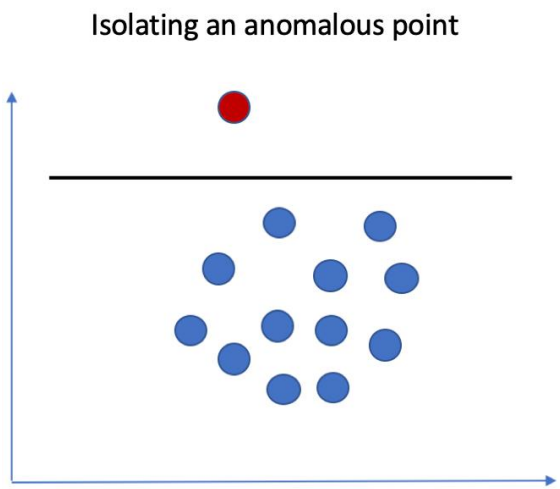
Grundlagen

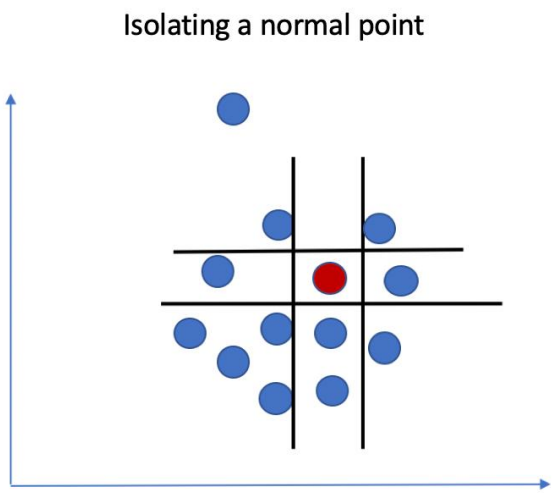
- **Speichereffizienz:** Die Baumstrukturen sind im Vergleich zu tiefen neuronalen Netzen oder großen Boosting-Ensembles extrem leichtgewichtig.
- **Benchmark für Unsupervised Learning:** In realen IoT-Szenarien sind gelabelte Angriffsdaten oft Mangelware. Der IF dient als Testlauf, wie gut ein System ohne Vorwissen über neue Bedrohungen funktioniert.
- **Inferenz-Geschwindigkeit:** Durch die geringe Rechentiefe eignet sich der IF ideal für Echtzeit-Analyse direkt auf Edge-Gateways.
- **Anomalie vs. Signatur:** Während CatBoost (supervised) darauf trainiert wird, bekannte Muster wiederzuerkennen, soll der IF das Potenzial haben, unbekannte Abweichungen (Zero-Day-Exploits) rein durch strukturelle Isolation zu finden.
- **Skalierbarkeit:** Der Algorithmus hat eine lineare Zeitkomplexität und ist damit prädestiniert für die riesigen Datenmengen, die im IoT-Umfeld (ToN-IoT Datensatz) anfallen.

Isolation Forest

Grundlagen



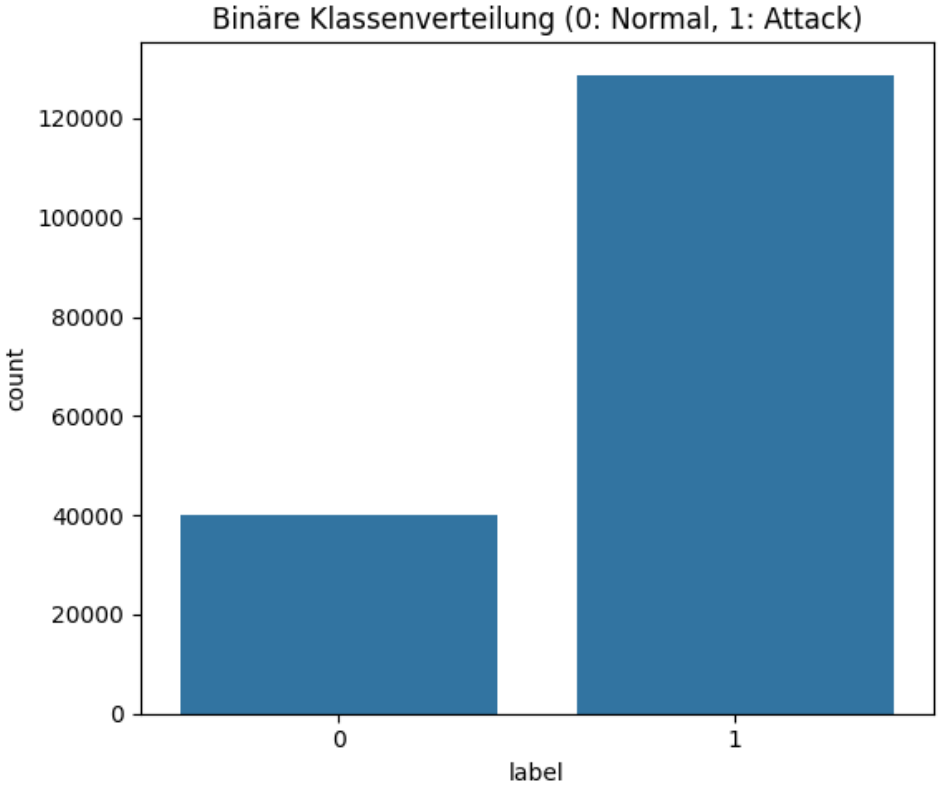




Isolation Forest

Besonderheit von TON-IoT

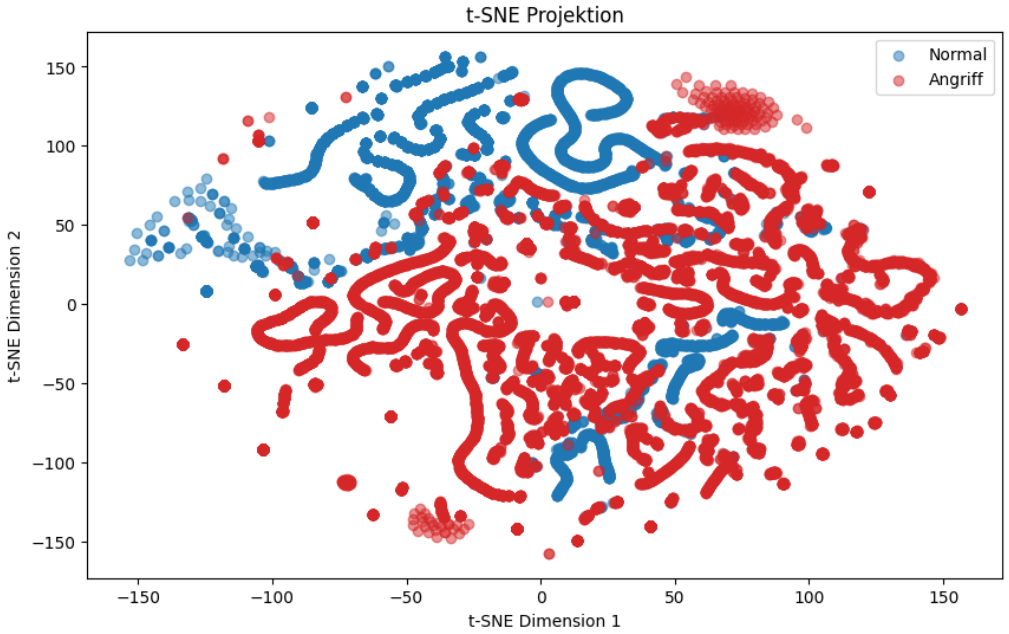
1. **Standard-Annahme:** Anomalien sind „wenig und anders“ (leicht zu isolieren).
2. **Realität im ToN-IoT:** Angriffe überwiegen (Mehrheit) und ähneln dem Normalverkehr.
3. **Das Risiko:** Hohe Rate an False Negatives, da das Modell Angriffe als „normal“ lernt.



Isolation Forest

Besonderheit von TON-IoT

1. **Standard-Annahme:** Anomalien sind „wenig und anders“ (leicht zu isolieren).
2. **Realität im ToN-IoT:** Angriffe überwiegen (Mehrheit) und ähneln dem Normalverkehr.
3. **Das Risiko:** Hohe Rate an False Negatives, da das Modell Angriffe als „normal“ lernt.



Isolation Forest

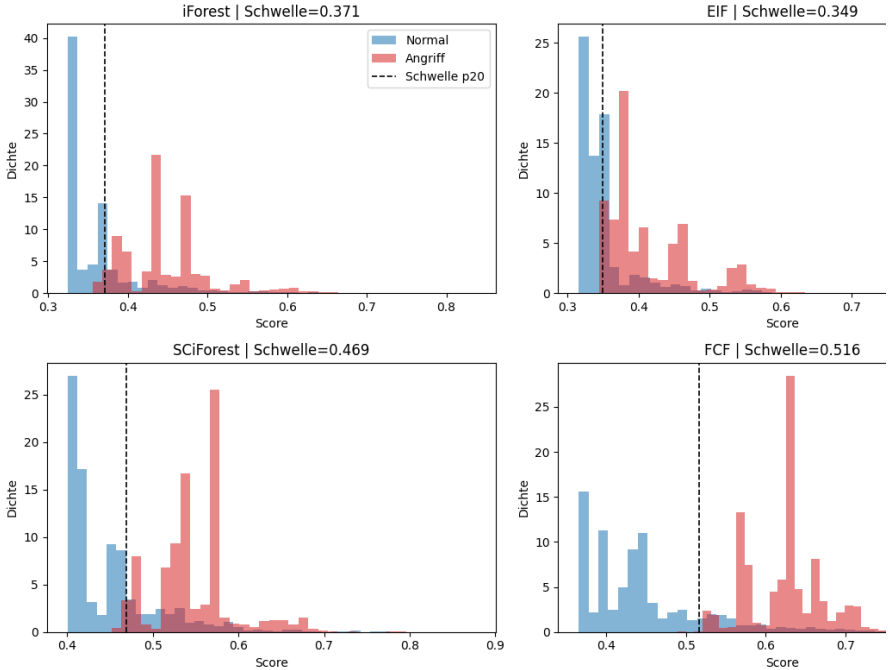
Bewältigungsstrategien

- **Reines Training auf Normaldaten:** Um das unüberwachte Paradigma zu wahren, wurde ausschließlich mit gutartigen Daten gelernt.
- **Einsatz von Architektur-Varianten, die über einfache Achsen-Splits hinausgehen:**
  - Extended IF (EIF): Verwendet Hyperebenen mit zufälligen Neigungen
  - SCiForest: Nutzt Hyperebenen, um Partitionen mit möglichst geringer Varianz zu erzeugen → Anomaliegruppen
  - Fair Cut Forest (FCF): Ziel sind balancierte Partitionierungen, korrigiert Ghost Regions von SCiForest
- **Parametrierung:** Erhöhung der `sample_size` auf 2048, um die heterogene Struktur des Netzwerkverkehrs besser zu erfassen
- **Preprocessing:** Ein Großteil der Spalten sind kategorisch und haben kaum Varianz → Presence- und Frequency-Encoding, Reduzierung der Features
- **Gezieltes Feature-Engineering:** Nutzung von EDA zur Entwicklung von trennscharfen Features
- **Semi-supervised Hyperparameter-Tuning:** Experimenteller Ansatz zur Optimierung der Hyperparameter

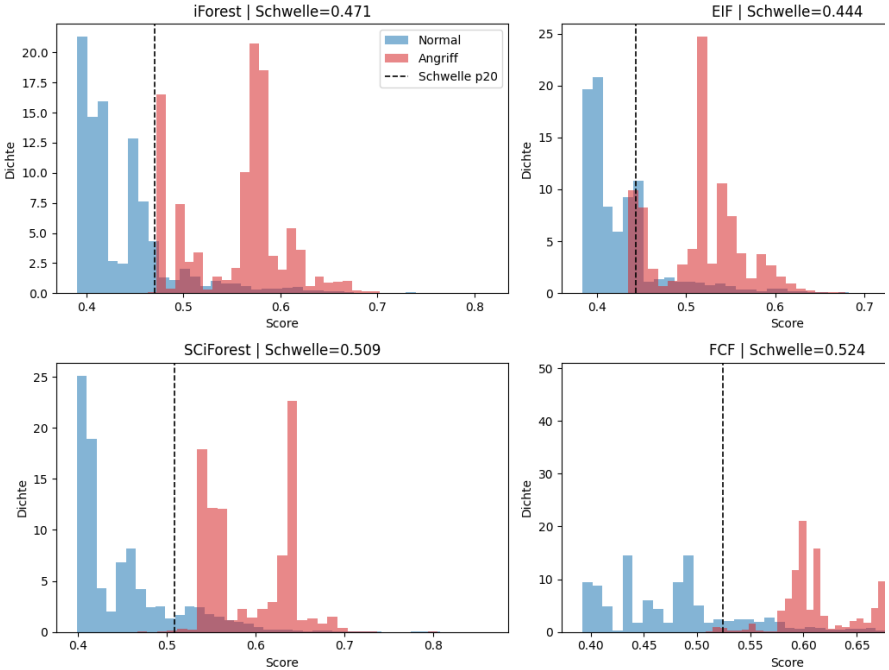
Isolation Forest

Feature Engineering

- Um die Klassentrennung im Training zu verbessern, wurden Verhältniswerte eingeführt, wie zum Beispiel:
- $\text{ratio_pkts} = \frac{\text{src_pkts}}{\text{dst_pkts}+\epsilon}$ Erfasst Asymmetrien im Paketfluss.
- $\text{mean_bytes_src} = \frac{\text{src_bytes}}{\text{src_pkts}+\epsilon}$ Charakterisiert die durchschnittliche Paketgröße.



Standard Features (vorverarbeitet)

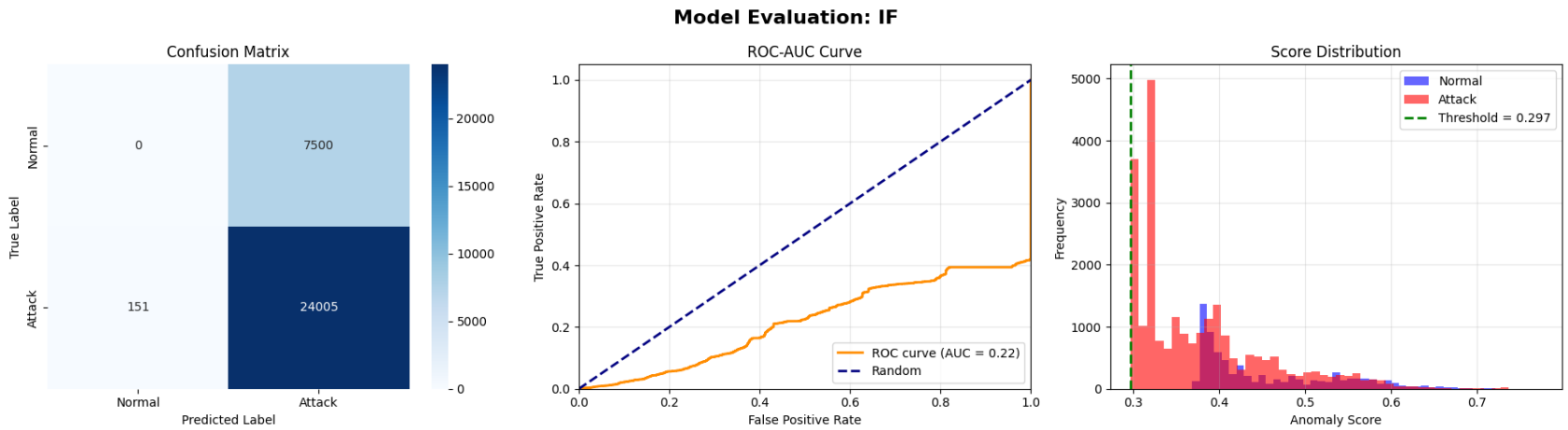


Konstruierte Features

Isolation Forest

Experiment: Semi-supervised Hyperparameter-Tuning

- **Ansatz:** Training auf Normaldaten, aber Tuning via Optuna (TPE-Sampler) unter Nutzung der Labels im Validierungs-Set.
- **Ziel:** Maximierung der Median-Differenz der Scores zwischen Normal- und Angriffsdaten.
- **Erkenntnis: Geringe Generalisierung.** Die Optimierung ist wahrscheinlich zu stark auf die spezifische Verteilung der Angriffe im Validierungsset fixiert (Overfitting auf die Angriffs-Charakteristik).



TON-IoT Intrusion Detection

Übersicht

01 Zielsetzung

02 Isolation Forest

03 **CatBoost Classifier**

04 Support Vector Machine

05 Evaluation

CatBoost Classifier

Grundlagen

OTH Amberg-Weiden

1. Warum CatBoost?

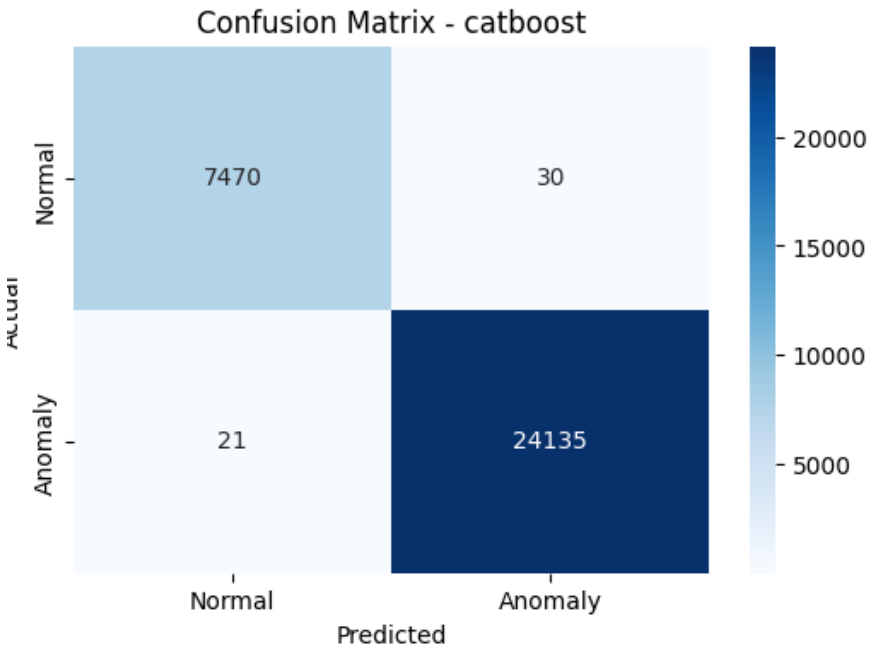
- **Effizienz:** Native Verarbeitung kategorischer Features (z. B. proto, service) ohne speicherintensives One-Hot-Encoding.
- **Robustheit:** Leistungsstarkes Gradient Boosting, das wenig Vorverarbeitung benötigt.

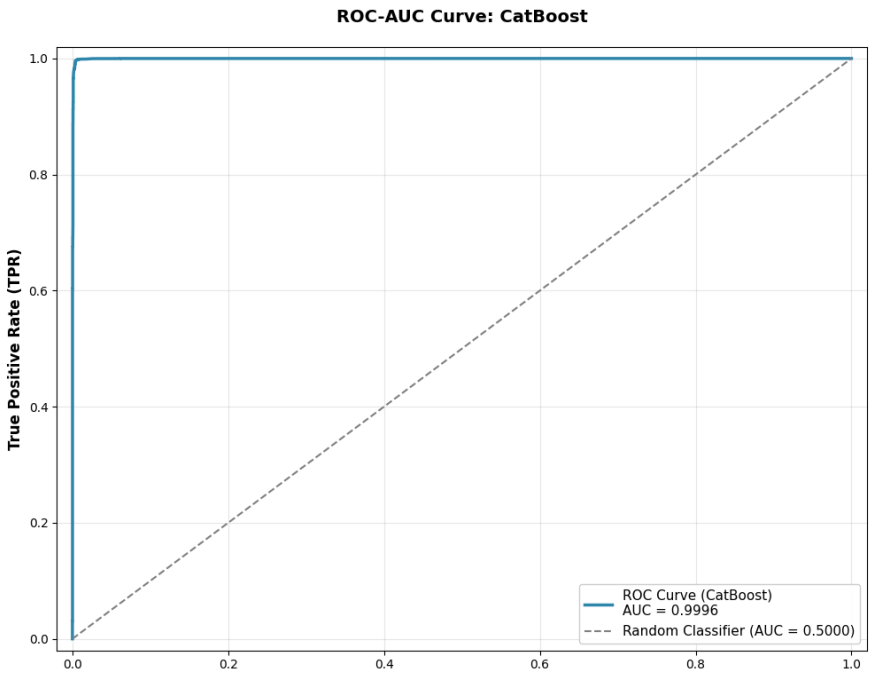
2. Optimierung & Tuning

- **Verfahren:** Bayes'sche Optimierung via Optuna (3-Fold Cross-Validation).
- **Parameter:** Tuning von Lernrate, Baumtiefe und L2-Regularisierung; dynamisches Early Stopping für die Anzahl der Bäume.
- **Vorverarbeitung:** Nahezu keine Vorverarbeitung, lediglich Markierung der kategorischen Features

CatBoost Classifier

Performance auf dem Evaluationsset





TON-IoT Intrusion Detection

Übersicht

OTH Amberg-Weiden

01 Zielsetzung

02 Isolation Forest

03 CatBoost Classifier

04 **Support Vector Machine**

05 Evaluation

TON-IoT Intrusion Detection

Übersicht

01 Zielsetzung

02 Isolation Forest

03 CatBoost Classifier

04 Support Vector Machine

05 **Evaluation**

Evaluation

Testsysteme

Komponente	System 1	System 2
CPU	AMD Ryzen 9 3900X (12 Cores, 24 Threads)	AMD Ryzen 5 8645HS (6 Cores, 12 Threads)
RAM	64 GB	16GB
OS	Ubuntu 22.04.5 LTS	Windows 11
Python	3.12.11	3.11.9

Evaluation

Testsysteme

Modell	Predicted Normal		Predicted Attack	
	TN	FN	FP	TP
SVC	8	0	7.492	24.157
LinearSVC	6.257	304	1.243	23.853
SGDClassifier	1.758	8.271	5.742	15.886
OneClassSVM	4.382	2.732	3.118	21.425
IF	3.504	4.208	3.996	19.949
EIF	4.625	1.927	2.875	22.230
SCIF	4.634	2.320	2.866	21.837
FCF	4.641	2.468	2.859	21.689
Catboost	7.469	24	31	24.133

Evaluation

Testsysteme

Modell	Inference Time (s)	Throughput (samples/s)	Peak RAM (MB)	Energy (kWh)	Peak CPU (%)
SVC	276,34	114,56	700,97	0,00064	11,7
LinearSVC	5,36	5906,42	551,98	0,00001	1,6
SGDClassifier	5,31	5963,27	611,89	0,00001	2,2
OneClassSVM	25,61	1236,35	435,89	0,00005	5,9
IF	5,38	5881,31	551,04	0,00001	4,5
EIF	6,61	4792,06	420,14	0,00001	0,0
SCIF	5,36	5901,72	509,16	0,00001	1,8
FCF	5,34	5931,98	453,83	0,00001	2,8
Catboost	7,51	4214,13	1014,96	0,00002	6,2

OTH Amberg-Weiden

VIELEN DANK!